

2: cloud service and deployment models

Friday, 20 February 2026 2:47 pm

Terminologies

- Cloud: special IT environment that provides servers, storage, software and networking through the internet
- Container: lightweight package that includes application, required libraries, required dependencies e.g. docker containers
- IT resource: server, database, software, network, storage; can be physical (real machine) or virtual (cloud based)
- On premises: when a company buys its own servers and installs them inside its building and manages everything itself
- Cloud providers: company that provides cloud services e.g. AWS, Azure
- Cloud consumer: person or company that uses cloud services
- Cloud broker: a middleman between cloud provider and cloud consumer; they help choose best service, negotiate pricing and manage services
e.g. a consulting company managing AWS services for a bank
- Cloud resource administrator: manages cloud services, VM, databases and security; can be the company itself or the cloud provider

Infrastructure as a service (IaaS)

- Provides virtual servers, storage, networks, OS
- We manage the application, data and OS whereas cloud provider manages the hardware
e.g. AWS EC2

Platform as a service (PaaS)

- Provides ready to use platform, OS installed, database installed, development tools installed
- We manage our application code
- Why use PaaS?
 - Extend on premises system
 - Replace local systems
 - Build and sell your own cloud services
 - e.g. google app engine, azure app service

Software as a service (SaaS)

- Ready-made software available through internet
- We just use it and don't manage anything; provider manages everything
e.g. gmail, netflix

Comparison IaaS vs PaaS vs SaaS

- IaaS: control level is high as we manage most of the things
- PaaS: medium control as we manage the code only
- SaaS: low control as provider manages all

Cloud deployment models

- Through public cloud
 - Cloud resources shared over internet
- private cloud (on premises),
 - Cloud infrastructure used by only one company
e.g. government, banks
- hybrid cloud
 - Combination of public cloud + private cloud
 - Sensitive data: private cloud, website hosting: public cloud

Similarities between AWS and traditional IT

- Both have servers, storage, networking, security and databases
- Traditional IT- physical hardware whereas AWS- virtual hardware

Model	Example
On-Premises	Buying your own car
IaaS	Renting a car
PaaS	Hiring a car with driver
SaaS	Using Uber

3/4: core cloud services

Friday, 20 February 2026 3:48 pm

ISP's and cloud

- Internet service providers provide internet access e.g. ptcl, stormfiber
- Cloud works through ISP; first connect to internet and then access AWS services

Cloud data center technology

- Cloud companies like amazon web services have huge data centers
- Data center stores data, runs applications, contains thousands of servers
- Each data center has backup power, backup networking, security, separate building
- 50000-80000 physical servers inside one data center
- Virtualization
 - One physical server can act like many virtual servers
- Points of presence
 - AWS has global locations called point of presence
 - It includes edge locations and regional edge caches
 - Used with amazon cloudfront and amazon router 53 to reduce latency (Delay)
 - Regional edge caches store content with infrequent
- AWS infrastructure features
 - Elasticity
 - Automatically increase or decrease resources
e.g. if 1000 users visit your website, AWS automatically increases servers
 - Scalability
 - Ability to grow when business grows
 - Fault tolerance
 - System continues working even if one part fails
 - High availability
 - System works almost all the time, minimal downtime

Core cloud services

1. Compute: running applications
 - a. Amazon EC2: virtual machine
 - b. AWS lambda: run code without managing servers
 - c. Amazon ECS: run containers
 - d. Amazon EKS: managed kubernetes
 - e. AWS fargate
 - f. AWS elastic beanstalk
 - g. AWS batch: runs hundreds of thousand of batch workload
2. Storage
 - a. Amazon S3: used to store files, images and videos; max single file=5TB, durable; used for backups, website hosting, media storage; object level storage (have to reupload entire changed file); storage classes can be standard (frequently accessed), intelligent-tiering, standard IA, one zone IA, glacier or glacier deep archive
 - b. Amazon EBS: hard disk for virtual machine; non-volatile (start/stop with all data intact), elasticity (increases capacity); can be encrypted; replicated in availability zone; used for OS disk, databases; it offers block level storage
 - c. Amazon EFS: shared storage for multiple servers
 - d. Amazon s3 glacier: for long term backup; very cheap and slow access
3. Networking
 - a. Amazon VPC: private network inside AWS
 - b. Elastic load balancing: distributes traffic to multiple servers
4. Databases

- a. Amazon RDS: for SQL databases
- b. Amazon DynamoDB: NoSQL database
- c. Amazon aurora
- d. Amazon redshift
- 5. Security
 - a. AWS identity and access management: controls who can access what
- 6. Cost management
- 7. Management and governance

Choosing a service

- Use EC2 when you want full control
- Use lambda when you don't want to manage servers

Three ways to use AWS

- AWS management console: graphical interface
- AWS CLI: commands
- SDK: use inside programming language

Amazon EC2

- Provides virtual machines (server) in the cloud
- Can be used as web server, database server
- We control windows or linux, choose size and region
- It is launched using amazon machine image (AMI)
- Steps to launch EC2
 - Select AMI template
 - contains OS and pre-installed software
 - Quick start (AWS provides), my AMI, marketplace (paid third party), community AMIs (shared by others), community
 - Select instance type
 - Decides cpu power, RAM, storage, network speed
 - General purpose, compute optimized (heavy cpu), memory optimized (big RAM), storage optimized, gpu/accelerated computing
 - Specify network settings
 - Vpc, subnet and public/private IP
 - Attach IAM role
 - Gives permission to ec2 to access other AWS services
 - User data script
 - We can add script that runs when server starts
 - Specify storage
 - Specify volume: size of disk, volume type, retain/delete, encryption or not
 - Block storage: split into fixed block so specific area is changed according to requirement; fast and reliable
 - Object storage: single portion of file cannot be modified; used to handle unstructured data; used for images, videos, backups
 - Add tags
 - They are labels assigned to AWS resource; consists of key and value
 - Helps organize resources
 - Security group settings
 - Set of firewall rules that control traffic
 - We decide which IP can access and which port is open
 - Key pair
 - Used for secure logins
 - Consists of private key that we keep and public key that is stored by AWS
 - For linux- SSH, windows- Remote desktop password
- Ec2 can be pending, running, stopped or terminated

5: cloud-native features

Friday, 20 February 2026 10:53 pm

Cloud native

- Applications that are designed specifically to run in the cloud
- They use microservices, containers, DevOps practices and focus on scalability and resilience
- Features
 - Scalability
 - Ability to handle more users or more data
 - Horizontal scaling: add more servers
 - Vertical scaling: increase power of one server
 - Supports growing users, improves performance and saves cost
 - Automated load balancing
 - Distributes traffic across multiple servers to avoid one server being overloaded
 - Types of load balancers
 - Layer 4: works at network level (Tcp/udp)
 - Layer 7: works at application level (http/https)
 - High availability
 - Application runs almost all the time
 - Achieved by redundancy, multiple servers, multiple regions
 - Secure by design
 - IAM controls who can access what and what actions can they perform
 - DevOps integration
 - Development means writing code, creating software and building applications
 - Types of development
 - frontend
 - Backend
 - Full stack
 - Role of a developer
 - Developer does coding, testing, debugging, problem solving, documentation, collaboration
 - Operations team deploys app, manages server, maintains infrastructure, monitors application
 - Application release process
 - Idea
 - Requirements
 - Code
 - Test
 - Build and package
 - Deploy
 - After deployment monitor, fix bugs and improve features to deliver application to users fast and safely
- Fast, reliable and easy to improve

Problems between dev and ops

- Miscommunication
 - Developer doesn't know deployment environment and ops doesn't understand code
- Conflict of interest
 - Developer adds new features quickly whereas operations maintain stability but main goal is same to deliver high quality apps to end users asap
- Security issues
- Testing issues

- Manual testing is slow; hard to test in multiple environments
- Manual work
 - Manual deployment, manual configuration, manual security checks slows the process, is error prone and hard to track changes

Solution: DevOps

- Combination of tools, culture, processes and goal is to automate and fast delivery
- DevOps culture focuses on collaboration (work together as a team), communication (Share info clearly and regularly), integration (combine and test code frequently) and continuous learning (always improve after mistakes) and connects developers, operations, QA and security
- DevOps processes include continuous integration where developers merge code frequently and code is automatically tested, continuous deployment where code is automatically deployed after testing, infrastructure as code (IaC), monitoring and logging to track errors, performance and usage.
- DevOps core values are automation, lean processes, measurement and sharing knowledge
- DevOps lifecycle tools are project management (Trello, jira), code (github), build tools (maven, npm), testing, CI/CD, containers (Docker, kubernetes), monitoring
- 7C's of DevOps
 - Continuous development
 - Continuous integration
 - Continuous testing
 - Continuous deployment
 - Continuous feedback
 - Continuous monitoring
 - Continuous operations
- DevOps teams are responsible for design, development, deployment, monitoring, maintenance
- End to end responsibility
 - Automation: use tools to automate everything
 - Cultural shift: shared responsibility
 - Continuous learning: review mistakes and then improve processes

6: emerging technologies

Saturday, 21 February 2026 3:31 pm

- Technologies are designed to run on cloud
 - IoT
 - Physical devices connected to internet
 - They collect data, send data to cloud and cloud processes and analyze it
 - Platforms include AWS IoT Core, azure, google cloud
 - They have devices, network, cloud platform and mobile/web application
 - Device -> cloud IoT hub -> database -> analytics -> visualization dashboard
 - Cloud databases store IoT data for analysis
 - AI/ML
 - It needs huge computing power, large datasets, GPU
 - Cloud provides powerful servers, GPU, managed ML services
 - Big Data
 - Large amount of data
 - Cloud helps in storing massive datasets and processing large data quickly
 - Tools used hadoop and spark
 - Blockchain
 - Digital ledger that stores transactions and cannot be easily changed and also it is decentralized
 - Used in cryptocurrency, smart contracts
 - Cloud provides blockchain as a service (BaaS)
- Cloud is essential because it provides scalability, global access, reduces cost and integrates all technologies

7: cloud-based ML workspace

Saturday, 21 February 2026 7:10 pm

Cloud for ML

- Scalability: increase computing power anytime
- Cost effectiveness: no need to buy expensive servers
- Collaboration: team members can access same cloud workspace
- Access to GPU/TPU: ML needs powerful hardware
- AWS, google cloud, azure

Microservices

- Software design style
- Instead of building one big application, break it into small independent services
- Each service has its own function, can be developed separately, can be deployed separately
- Monolith means all modules in one application
- Microservices are usually deployed in containers where each service runs inside its own container; increases flexibility
- Architecture
 - Client -> API gateway -> services -> database
 - Each service have its own database

Containerization

- Container: lightweight virtual environment
- It contains application, dependencies and libraries
- Containerization: process of packaging application into container; it works everywhere
- Tool used is docker

Bare metal vs VM vs container

- Bare metal: install app directly on physical server
- Virtual machine: each VM has its own OS, uses fixed memory so memory wastage
- Containers: share same OS and use only required memory
- Benefits
 - Portability: Build once and run anywhere
e.g. build ML model in laptop container, deploy same container to AWS; no compatibility issues
 - Scalability: easily add more containers when demand increases
 - Efficiency&speed: containers start quickly, use less memory and deploys fast; helps in CI/CD pipeline

Cloud workspace architecture

- Upload dataset to cloud storage
- Start compute instance
- Train model
- Save trained model
- Deploy model as API
- Monitor performance

Example Complete Flow:

Student trains image classification model:

- Dataset stored in S3
- GPU EC2 instance used
- Model trained
- Model deployed using container
- Users send image → API returns prediction

Everything runs in cloud.

8: containers&dockers

Saturday, 21 February 2026 8:09 pm

Docker desktop

- It is a complete tool for running docker on my computer
- Includes
 - Docker engine: runs containers
 - GUI: manage containers visually
 - Docker compose: run multiple containers together
 - Kubernetes: test container orchestration
 - Volume and file sharing: share files between system and orchestration

Docker engine

- Works as a server (Daemon called dockerd), API, command line interface

Command:	docker run image_name
----------	-----------------------

Docker engine creates container, allocate resources and runs applications

Docker

- It is a platform to build, ship and run applications in containers
 - It packages code, libraries, dependencies and runtime into one container
 - It ensures same behaviour in all environments, fast deployment and better resource usage
 - Examples
 - Model training environment: whole team uses same environment so no version conflicts
 - Model serving
 - Pipeline integration: data preprocessing, feature engineering, training, evaluation, deployment each step can run in separate container; tools like airflow, mlflow manage these containers
 - CI/CD for ML: docker images are used in jenkins which automatically test model, train model and deploy model
- | | |
|---------------|--------------------------|
| Build image | Docker build -t ml-train |
| Run container | Docker run ml-train |
- Docker architecture
 - Type command
 - CLI send request
 - Daemon build image
 - Container runs
 - Output shown to user
 - Without docker install dependencies manually, different versions on different machines, hard to manage environments

MLOps

- ML lifecycle includes data collection, training, validation, deployment, monitoring and retraining
- DevOps focus on application code, continuous integration and continuous deployment whereas MLOps focus on data versioning, model versioning, experiment tracking, model monitoring, retraining pipelines

9: orchestration, containers, dockers

Saturday, 21 February 2026 8:35 pm

Orchestration

- Managing ML workflows and containers automatically
- Includes
 - Provisioning resources (CPU,GPU)
 - Scheduling tasks (preprocessing before training)
 - Scaling containers up/down
 - Monitoring services
 - Automating pipelines
- Examples of orchestration tools
 - Kubernetes: manages docker container
 - Apache airflow: manages workflow (Task pipelines)
 - Kubeflow: (ML workflows on kubernetes)
 - Mlflow: (used for ML CI/CD automation)
- Docker vs orchestration
 - Docker runs one container
 - Orchestration comes in when you want preprocessing first, then training, then deployment, automatically every day with scaling
- Orchestration with airflow
 - Airflow uses directed acyclic graph (DAG) where the order is defined as preprocess-train-deploy



ML Model Lifecycle

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG("ml pipeline", start_date=datetime(2025,1,1), schedule_interval="@daily") as dag:

    preprocess = BashOperator(
        task_id="preprocess",
        bash_command="docker run data-preprocess"
    )

    train = BashOperator(
        task_id="train",
        bash_command="docker run model-train"
    )

    serve = BashOperator(
        task_id="serve",
        bash_command="docker run -d -p 8000:8000 model-api"
    )

    preprocess >> train >> serve
```

- Orchestration with kubernetes
 - Kubernetes ensures 3 copies of model API, auto restarts if one crashes and load balance traffic



ML Model Lifecycle

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: model-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: model-api
  template:
    metadata:
      labels:
        app: model-api
    spec:
      containers:
        - name: model-api
          image: model-api:latest
          ports:
            - containerPort: 8000
```

- Docker vs kubernetes vs airflow
 - Docker runs single container
 - Kubernetes manages and scales many containers
 - Airflow manages workflow steps and scheduling

10: docker

Saturday, 21 February 2026 9:08 pm

Running containers list	Docker ps
Running and stopped containers list	Docker ps -a
Shows images (image name, tag, size)	Docker images
• Run a container	Docker run hello-world
Remove containers	Docker rm container-name
Remove images	Docker rmi image-name
Build docker image	Docker build -t imagename

- Container lifecycle
 - Creat -> run -> pause -> stop -> restart -> remove
- Dockerfile
 - A textfile that contains instructions to build docker image
 - It tells docker which base image to use, what files to copy, what commands to run and what program to start
 - Layered architecture
 - Docker images are built in layers and each instruction creates a new layer
 - If we change one layer, then docker rebuilds only that layer
- Docker compose
 - Tool for running multiple containers together
 - It defines multiple services, manages network, manages volume, builds image and starts containers
 - Docker compose file.yaml

Docker Compose

File: compose.yaml

```
services:
  web:
    build: .
    ports:
      - "8000:5000"
  redis:
    image: "redis:alpine"
```

- This means build web app from dockerfile, run redis container and connect both
- Run everything using Docker-compose up
- Consider it as an octopus where each arm- one container that controls building images, pulling images and running containers automatically
- Container build and run process
 - Write dockerfile
 - Build image

- Run container
- Expose port
- Application runs
- Storage and volumes
 - Containers are ephemeral meaning when container is delete, data is lost
 - The database inside container loses data so here is where volumes comes in
 - Volumes are storage managed by docker, stored outside container, so even if container is remove, data is still safe in volume
 - Bind mounts connect specific folder on my computer to folder inside container

11: versioning

Saturday, 21 February 2026 10:00 pm

Dockerfile: a textfile with instructions to build an image

Image: a read only template build from dockerfile

Container: running instance of an image

Dockerfile instructions

- FROM: set base image
- COPY: copy files
- RUN: execute commands
- CMD: run program

Model versioning

- ML models change over time because new data arrives, model retrained and hyperparameters changed so if we don't version then we cannot track which model was deployed
- Versioning strategies
 - Tagging e.g. v1,v2,v3
 - Semantic versioning
 - Metadata e.g. attach info like dataset id, git commit id and training date

Why orchestration

- It automates scheduling retraining, running pipeline and deploying new model

Why registries

- It is a storage place for docker image
- Needed for team collaboration, kubernetes deployment and central storage

Why monitoring and rollback

- After deployment we must monitor accuracy, latency, error rate
- Rollback means return to previous stable version

12: TensorFlow Extended (TFX)

Wednesday, 8 April 2026 3:26 pm

Challenges

- Building ML systems in real life is hard because of different ML tasks, continuous training needed, human involvement and need for scalability and reliability

What is TFX?

- Without TFX:
 - Systems are messy
 - Too many components
 - Hard to manage
- With TFX:
 - It is a platform to build complete ML pipelines
 - It organizes all ML steps and works with tools like apache airflow, kubeflow
 - Instead of writing separate scripts for data cleaning, training and deployment, TFX manages everything in one pipeline
 - A model trained once can be retrained automatically, deployed safely and updated regularly
 - Research to production with stability, consistency and repeatability

When to use TFX?

- For large datasets, need consistency between training and serving and for version control

TFX Building Blocks

- Layer1: data and execution (apache beam for big data processing)
- Layer2: core components
- Layer3: orchestration (controls flow)

Core Components

- Data analysis -> data validation -> data transformation -> model training -> model evaluation -> model serving

TFX Data

- Data errors are hard to detect and cause model to degrade slowly
- So TFX uses 3 steps that are analysis (to understand data) -> transformation (to prepare data) -> validation (check correctness)

TFX Data Analysis

- Goal is to understand data using statistics so it checks missing values, distributions and correlations

TFX Data Transformation

- Goal is to prepare features for model and convert data into usable format
- It stores transformation logic so it is same at the time of training vs production

TFX Data Validation

- Goal is to check if data is correct so uses version schema
- It defines feature type, allowed values and required fields

TFX Model Training

- Goal is to train model efficiently
- Any model can be configured using tensorflow and easy to switch between algorithms without rewriting pipeline
- Problem is large datasets take days for training
- Trade-off is if fast training then low quality and vice versa
- Solution:
 - warm start
 - Instead of training from scratch, begin from previous model as old model knows user preferences hence faster training and same quality
 - High level APIs for productivity

- Use tools such as `featurecolumns` that define input features and estimator that handles training

TFX Evaluation and Validation

- Problem is that model may look good but fail in real world and the goal is to build a reusable component to automatically evaluate and validate models before they are served
- Model must not crash on bad input and predict correctly

TFX Model Serving

- Goal is to serve model efficiently
- It requires it to be fast, scalable and reliable
- Problem 1
 - Loading model slows system
 - Solution: use separate threads
- Problem 2
 - Data parsing is slow
 - Solution: optimized protocol buffer parser

TFX Impact

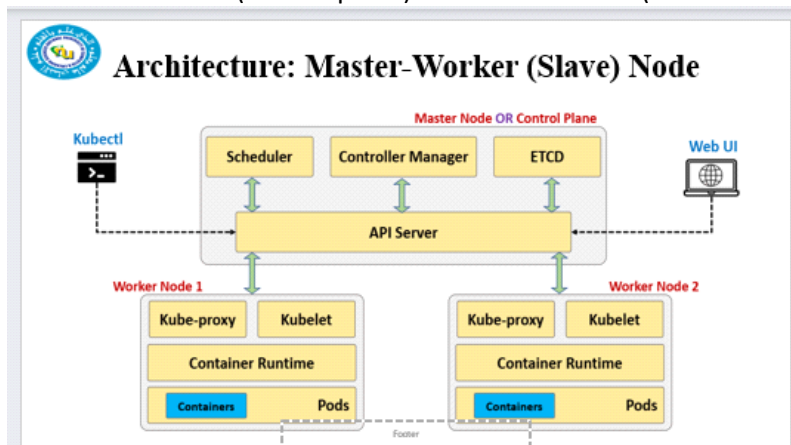
- For developers:
 - Faster development and less code
- For business
 - Better models and more revenue

13: Orchestration using Kubernetes

Sunday, 12 April 2026 2:22 pm

Kubernetes

- A system that helps you deploy applications, scale them i.e. increase or decrease resources and manage them automatically
- Used for containerized applications like docker
- It also restarts container if they crash
- Orchestration means managing multiple containers
- Kubernetes is needed because it can handle many containers unlike previous apps that were monolithic (single large system)
- Benefits
 - High availability: always running
 - Scalability: increase/decrease containers automatically
 - Disaster recovery: if system fails, kubernetes restores it
- Features
 - Automated deployment
 - Auto scaling
 - Auto restart
 - Load balancing - if one server is overloaded, it send traffic to another
 - Storage management
- Architecture
 - Has a master node (control plane) and worker nodes (where actual app runs)



- Master node components
 - Api server
 - ◻ Entry point of kubernetes, all requests go through it
 - Etcd
 - ◻ It is the database of kubernetes and stores cluster state
 - ◻ Example: which pods are running and how many replicas exist
 - Controller manager
 - ◻ Watches system state and fixes issues automatically
 - Scheduler
 - ◻ Decides where to run the container i.e. which worker node has enough memory
- Worker node components
 - Kubelet
 - ◻ Communicates with the master and ensures containers are running
 - Container runtime
 - ◻ Runs containers

- Kube-proxy
 - It handles the networking and sends traffic to correct container
- Kubernetes core components
 - Pod
 - It is the smallest unit in kubernetes and contains one or more containers
 - They share network and storage
 - Service
 - It connects the pods and provides stable access so even if pods change, service keeps same address
 - Ingress
 - Allows users to access app from internet
- Kubernetes objects
 - The things that kubernetes manage are called objects, they include pod, deployment and service
 - Spec vs status
 - Spec is the desired state (what u want)
 - Status is the current status (what is happening rn)
 - Replicaset ensures required number of pods always run
- Minikube
 - It is a local kubernete setup that runs on ur device
- Kubectl
 - A command line tool that is used to deploy apps, check status and debug

14: Scalable Orchestration

Sunday, 12 April 2026 3:14 pm

Challenges

- Deployment friction
 - Manual process so slow updates
- Endpoint proliferation
 - Each model has its own API that is there are many endpoints and hard to manage
- Monitoring gaps
 - Hard to track which models failed or why performance dropped
- Governance issue
 - Hard to track who created the model and which version is deployed
- Resource inefficiency
 - Cpu/gpu is not used properly

Solution: unified orchestration hub

- One central system to manage all models, one API for all models and handles routing automatically

Architecture overview

- API gateway
 - Entry point for all users that handles authentication and security
- Orchestration service
 - Brain of system that decides which model to call and how to combine results
- Model microservices
 - Each model runs independently
- Kubernetes platform
 - It handles scaling, load balancing and self healing
- Supporting services
 - Redis for caching, database for storing configurations and monitoring tools

Workflow

- User sends a request -> API gateway receives it -> request goes to the orchestrator -> orchestrator checks rules and calls multiple models -> models return results -> orchestrator combines results -> final response sent to user

Scalability and high availability

- Replication
 - Multiple copies of same service
- Load balancing
 - Traffic is distributed evenly
- Health checks
 - Kubernetes checks if container is still alive, if not then restart
- Autoscaling
 - Automatically increase pods when load increases
- Distribution
 - Pods are placed on different machines to prevent full system crash

MLOps benefits

- Automated deployment
- Standardized serving
- Observability
- Experimentation
- Easy rollbacks
- Governance

Implementation considerations

- Security
 - API gateway must protect system
- Testing
 - Automated testing pipelines
- Monitoring
 - Need full observability tools
- Performance
 - Orchestrator must be fast
- Change management
 - Switching to unified API may require changes

Trade-offs and risks

- Bottleneck risk
 - Orchestrator can slow system
- Single point of logic
 - All decisions centralized
- Complexity
 - Hard to debug distributed system
- Requires expertise
 - Needs skilled DevOps/MLOps team

Connection with TFX

- TFX builds ML pipeline and handles data, training and validation
- What kubernetes orchestration does is it deploys models, scales models and manages multiple models
- TFX trains model, kubernetes deploys it and orchestrator manages it with others
- Without TFX models may be of poor quality
- TFX ensures model correctness and reproducibility and kubernetes orchestration ensures scalability, resilience and governance

15: Kubeflow

Sunday, 12 April 2026 4:08 pm

- Kubeflow is an open source platform used to build ML workflows, deploy models, manage pipelines and scale using kubernetes
- Kubeflow=ML+kubernetes

Kubeflow vs airflow

- Airflow is a general workflow tool and not designed specifically for ML
- Kubeflow is designed for ML workflows and supports training, tuning and serving

Importance of ML pipelines

- A pipeline is a sequence of steps, they are important because of automation, reproducibility, scalability and collaboration

Kubeflow ecosystem

- Kubeflow has many tools
 - Kubeflow pipelines
 - Notebooks
 - Training operators
 - Kserve
- Write code in notebook, train using operators and deploy using Kserve

Why kubeflow on AWS?

- AWS helps because EKS manage kubernetes, S3 for data storage, EC2 for compute power and CloudWatch for monitoring

Pipeline components and DSL

- DSL is domain specific language, it is a python based way to define pipelines
- We write python code that defines steps and kubeflow converts it into kubernetes workflow

Writing python components

- A component is a single step in pipeline
 - Example Component:
 - `@dsl.component`

```
def preprocess_data(input_path: str, output_path: str):  
    import pandas as pd  
    df = pd.read_csv(input_path)  
    df = df.dropna()  
    df.to_csv(output_path, index=False)
```

@dsl.component vs @dsl.pipeline

- `@dsl.component` defines a single task and `@dsl.pipeline` defines a full workflow

Control flow

- Kubeflow supports
 - Conditions: only run if condition is true
 - Loops: run tasks multiple times
 - Caching: avoid re running same step
- `@dsl.pipeline(name='train-pipeline')`

```
def pipeline():  
    preprocess_op = preprocess_data()  
    train_op = train_model(preprocess_op.output)
```

AWS EKS architecture

- AWS manages control plane and we manage the worker nodes

Prerequisites

- IAM, VPC, S3, ECR
- Steps:
 - Create eks cluster, configure kubectl, deploy kubeflow, check pods, setup ingress

STEPS



Combining Components

```
@dsl.pipeline(name='training-pipeline')
def pipeline():
    ingest = ingest_data()
    preprocess = preprocess_data(ingest.output)
    train = train_model(preprocess.output)
    evaluate = evaluate_model(train.output)
```



Executing the Pipeline

- Compile the pipeline and submit to Kubeflow

```
from kfp import Client
client = Client()
client.create_run_from_pipeline_func(pipeline,
arguments={})
```

16: Deploying a model

Sunday, 12 April 2026 4:33 pm

- Model as a function/library
 - Simple usage inside code
 - Example: `model.predict()`
- Model as a service
 - Exposed via API
 - Can be used by other systems
- Packaging in a container
 - Use docker
 - Makes deployment portable
- Deploy in cloud
 - Accessible globally
- Auto scaling
 - System handles high traffic automatically
 - Stateless serving functions pattern
- MLOps tools
- Looks easy when broken into above steps, but real world deployment is complex, challenges include offline usage, scaling issue, hardware cost and frequent model updates

Where should the model live?

- Cloud (server), phone (client side) and edge devices

Considerations for deployment

- Input data size
- Output size
- Speed
- Energy consumption
- Model size
- Update frequency
- Cost
- Offline availability
- Monitoring
- Collaboration between teams
- Experimentation support
- Coupling of ML components

Deployment design types

- Static intelligence
 - Model fixed inside product
 - No updates
- Client-side intelligence
 - Runs on user device
 - Works offline
- Server-centric intelligence
 - Runs on cloud/server
- Back-end cached intelligence
 - Precomputed results stored
- Hybrid models
 - Combination of client+server

Where should feature encoding happen?

- Client or server side

- Client: faster, less server load
- Server: more control, consistency

Operational robustness

- Redundancy
 - Backup systems
- Load balancing
- Error handling
 - Prevent system crashes
- Logging and monitoring

Integrating models into a system

- ML models must integrate with business logic, API and databases
- A model is not the whole system
- Separate model and business logic for easier development, independent updates
- Document inputs, outputs, expected behaviour so others can use the model and others know what model does, its limitations etc
- Document accuracy, fairness, latency, reliability, ethical concerns

17: Edge Devices and Containers

Wednesday, 15 April 2026 9:15 pm

What is edge computing

- Running computation (AI/ML) close to where data is generated, instead of sending it to a cloud server
- Traditional method
 - Data is sent to cloud, processed in data center and result is sent back
 - Problems
 - Slow, needs internet and not reliable in remote area
- Edge computing
 - Data processed on local device
 - Faster response, is secure as it is stored locally and cost effective

What are edge devices

- Those devices that run ML models locally and perform inference (predictions) e.g. mobile phones, IoT devices
- Edge devices are useful where there is no internet, or at remote location e.g. self driving car must not wait for cloud response, it should decide instantly

Coral

- Platform for running ML models directly on devices
- Features
 - Fast inference
 - Works offline
 - Supports tensorflow lite models

Azure percept

- Platform by microsoft for edge devices, model deployment, cloud integration

TFHub

- A repo of pre-trained models since not all models work on edge devices, only compatible ones do because of limited memory and power.

18: Containers for Managed ML systems

Wednesday, 15 April 2026 9:28 pm

ML pipelines

- ML models do not last forever because data keeps on changing and patterns evolve so we train multiple models, use different parameters, compare results and select the best one amongst them
- Automation
 - Pipelines can run when data changes, when code changes, when parameter changes or they can continue running in a loop

ML pipeline stages

- Data storage and ingestion
 - Collect data from databases and store it in a cloud
- Data exploration and preparation
 - Clean data, remove errors and transform features
- Model development and training
 - Train ML model
- Multi stage execution tracking
 - Track each pipeline step
 - Tracks when training happened, which data was used, code version, packages, runtime environment etc
 - Types of testing
 - Data quality tests
 - Check missing values or incorrect data
 - Model performance test
 - Checks accuracy, precision, recall
 - Serving application test
 - Checks whether API is working correctly
 - Pipeline tests
 - Checks whether pipeline handles various exceptions

Managed ML systems

- Cloud based platforms that automate ML workflows, manage infrastructure, simplify deployment.
- System handles everything after u upload data
- Google cloud AI platform, AWS sagemaker
- Containers for managed ML systems
 - Containers package code + dependencies and can run anywhere

19: AWS SageMaker

Wednesday, 15 April 2026 9:44 pm

- Managed ML systems= cloud platforms that handle ML infrastructure automatically

AWS SageMaker

- A fully managed ML service by amazon
- It helps in building models, train models, deploy models
- Combines ML+DevOps
- MLOps pipeline in sagemaker
 - Collect and prepare data
 - Choose algorithm
 - Set training environment
 - Train and tune model
 - Deploy model
 - Scale and manage
- Objectives
 - Faster development
 - Better collaboration
 - Improved accuracy
 - Large scale processing
- Build phase
 - Use Pre built notebooks
 - Environment setup
 - Training + tuning
 - Build ur own algorithm using tensorflow,pytorch
 - Use of framework SDK
 - Built in algorithms
 - Kmeans, PCA, XGBoost etc
 - Deployment support
- Sagemaker workflows
 - Tool for ML pipelines
 - Features
 - Automates step
 - Scheduled runs
 - Integrates with kubernetes
 - Data -> prep -> train -> deploy -> monitor
 - All of this happens in sagemaker studio
- Sagemaker studio
 - It has notebooks, dataset management, training and deployment
 - e.g. jupyter + AWS combined
 - Data preparation
 - Processing, ground truth for labeling, feature store for reusable features
 - Model training
 - Built in algorithms, custom models,distributed training
 - Hyperparameter optimization
 - Automatically finds best parameters instead of guessing
 - Model deployment
 - Real time API, batch processing, multi model endpoints
- AWS integrations

- S3 for storage
- Lambda for automation
- Cloudwatch for monitoring
- ECR for container storage

20: CI/CD pipelines for machine learning

Thursday, 11 June 2026 12:06 pm

MLOPS Level 0: Manual Process

- Everything is manual and script driven
- Disconnection between development and operations
- Since everything is manual, new model versions are deployed rarely
- No CI/CD so changes are not automatically tested or deployed
- No performance monitoring even if it degrades over time
- Challenge: Models that work perfectly in development often fail in production because data distribution changes, deployment environment differs from the training environment. This is known as the "works on machine" problem

MLOPS Level 1: ML Pipeline Automation

- Rapid Experimentation
 - Pipeline is automated enough to quickly iterate on different models, features and hyperparameters without manually re-running every step
- Continuous training of the model in production
 - Instead of training once and deploying forever, the model is automatically trained on new data as it arrives. This keeps the model accurate
- Modularized code
 - Structured into reusable pipeline components
- Challenge: testing and deployment is still manual

CI/CD

- CI
 - Frequently merging code changes into a shared repository, with each merge automatically triggering a build and test process.
 - Goal is fast feedback, within minutes the bug can be identified
- CD
 - Validated code is automatically prepared for delivery to production- like environments and then deployed without human intervention
- ML CI/CD must handle data versioning, model training, evaluation, deployment, monitoring, retraining
- ML needs CI/CD for faster iteration, reproducibility and safer deployment

MLOPS Level 2: CI/CD

- Changes to pipeline components are automatically built, tested and deployed
- When new training data arrives, the entire pipeline from ingestion to serving automatically executes
- The whole system is monitored, failures trigger automated alerts and rollbacks
- Multiple teams can collaborate because everything is version controlled

JENKINS FOR MLOPS

- Primary CI/CD tool for implementing level 2 MLOPS
- It is an open source automation server that automates the software delivery pipeline, it integrates with GIT, runs automated tests
- Components
 - Jobs
 - Defines a task to run e.g. build docker image
 - Can be triggered automatically on code changes, on a schedule or manually
 - Each job has its own configuration
 - Pipelines
 - Multiple jobs chained together into an end to end workflow
 - From code commit to production deployment

- Write pipelines as code, which lives in your git repo and is version controlled
- Agents
 - Distributed servers where jenkins executes job.
 - It distributes work across multiple agents (VM, containers) and so it allows parallel execution and scales with workload
- Plugins
 - It has over 16000 plugins.
 - Key plugins integrate with Git (source control), docker (containerization), kubernetes (orchestration), slack
- Jenkins pipeline stages
 - Code
 - Developers write and commit code to a git repo
 - A webhook is configured so every new commit automatically notifies jenkins, triggering the pipeline
 - Build
 - Uses build tools to compile the source code, package it and run unit tests
 - Test
 - Integration test, UI test, performance tests, security tests using frameworks like selenium
 - Results, logs and metrics are collected
 - Signing/ security
 - Builds undergo code signing, vulnerability scanning, and manual approval gates for governance
 - Deploy
 - Validated code/ models are deployed to testing, staging, or production using containers
 - Inform
 - Teams receive automated notification with pipeline status, logs and reports

GITHUB ACTIONS FOR ML

- Automation built directly into github repositories. No separate server is needed
- advantages
 - Simple setup: only requires an additional .yml file in the repo and github handles all execution
 - Respond to any github webhook: can be triggered by any event that occurs in a github repository; pushing code or opening a pull request etc
 - Community powered, reusable workflow
 - Any platform, any language or any cloud
- Components
 - Workflows
 - Defines the complete automated process
 - A .yml file stored in the .github/workflows/ directory
 - It is version controlled- track changes, review it in PRs
 - Runs automatically when triggered by an event
 - Events
 - A specific activity that triggers a workflow to run
 - Push, pull request, release and schedule, workflow_dispatch
 - Jobs
 - Set of steps within a workflow that all execute on the same machine
 - Steps within a job execute sequentially and share the same file system
 - Multiple run within the same workflow can run in parallel by default
 - Actions
 - Pre-defined, reusable piece of automation that performs a specific task
 - Runners

- Server that executes the workflow when it is triggered
- Github-hosted runner (vm provided and managed by github) or self-hosted runner (useful for when you need GPU for model training, access to private networks/databases)

21: Monitoring, Logging and Scaling

Thursday, 11 June 2026 2:26 pm

Model Monitoring

- Continuously collecting, analyzing and visualizing data to detect
 - Data drift: when distribution of live input data starts to deviate from training data
 - Concept drift: when underlying relationship between inputs and outputs changes in the real world
 - Performance degradation: direct drop in model quality
- Metrics to monitor
 - System metrics: CPU usage, memory, disk I/O, network
 - Application metrics: API latency, error rate
 - Model metrics

Why monitoring matters

- Ensures model reliability
 - If a model silently degrades without detection, real damage can occur
- Detects performance drops early
 - Without monitoring you wouldn't know a model is failing until users complain
- Supports continuous improvement

Logging in ML

- While monitoring watches aggregated metrics, logging records individual events- specific API calls, errors, warnings
- Why logging is critical in ML
 - When a model makes a wrong prediction, logs makes it easier to see exactly what input it received
 - It tells which steps failed
 - It provides audit trail
- Logging levels- least to most critical
 - Debug: detailed development info-step by step tracing
 - Info: normal operation events
 - Warning: something unexpected but not breaking e.g. missing values detected
 - Error: failure occurred but system continues
 - Critical: system level failure, execution halting e.g. out of memory

```
logging.debug("This is a debug message - helpful for development.") logging.info("Model training started.")
logging.warning("Missing values detected, using default imputation.")
logging.error("Model failed to load - check file path.") logging.critical("System out of memory - halting execution.")
```

Tools

- Aws tools
 - Cloud watch: monitoring service
 - Sagemaker: detects data drift
- Azure tools
 - Azure monitor
 - Log analytics
- Open source
 - Prometheus: metrics collection and alerting system
 - Grafana: visualization and dashboard

- Mlflow

Prometheus

- Multi dimensional data model with time series
- Flexible query language: promQL
- No reliance on distributed storage
- Pull model over HTTP: prometheus actively scrapes metrics from targets via HTTP
- Service discovery
- Push gateway for short lived jobs
- Multiple dashboarding modes
- Components
 - Prometheus server
 - Responsible for scraping (actively pulling metrics from targets via HTTP) and storing
 - Special purpose exporters
 - Pre built metric collectors for common systems and services
 - e.g. node exporter,mysql exporter
 - Client libraries
 - For custom applications. You instrument your code using these libraries to expose custom metrics directly from your application
 - Alertmanager
 - Receives alerts fired by the prometheus server and handles routing
 - Push gateway
 - For short lived jobs such as model training that completes too quickly for prometheus to scrape them
- Prometheus: Data model
 - Every piece of data is a time series- a stream of timestamped values that share the same metric
 - Prometheus can also generate temporary derived time series during query execution e.g. computing requests per second
- Prometheus: monitoring
 - Prometheus server runs continuously and there are targets being monitored and scraping is done where prometheus collected metrics from targets
- Prometheus: exporters
 - An exporter is a small process that runs alongside a target system. It continuously fetches metrics from that system and re-exposes them in prometheus text format via an HTTP endpoint
- Prometheus: consul
 - A service registry tool where every service registers itself over here so instead of a static list of Ips, prometheus queries consul to dynamically discover what services are currently running and where they are
 - Allows you to register new instances automatically and de-register terminated ones
 - Handles failure such as if a target goes down
- Prometheus: alertmanager
 - Responsible for alerts
 - Routing: Different alerts are sent to different receivers,
 - De-duplication: and if same alert fires multiple times then alertmanager only send one notification
 - Grouping: It also groups multiple fires at once instead of sending separate messages

Grafana

- For analytics and monitoring
- Query (write promql to pull exactly the data you want), visualize, alert and understand